

SOLUTION

ECE 2036 Exam 3

Spring 2025

Instructions: You have 2 hours, 50 minutes to complete this exam (but you should not need the full time). The test is closed book and closed notes. No calculators, phones, or other electronic devices are allowed. Unless stated otherwise, justify your reasoning clearly to receive any partial credit. You must write your answer in the space provided on the exam paper itself. If space is needed for additional work, please use the backs of previous pages.

I. Fill in the blank

10 points

Good programming practice with object-oriented programming is to separate the interface from the implementation of a class so that other programmers know how to use the class without having to worry about its inner workings. In a class, attributes are implemented with data members and behaviors are implemented with member functions. In function calls, passing a variable by reference means that the variable in the calling routine will be modified if the parameter is modified inside the function while passing by value means the variable in the calling routine will not be affected by anything that happens inside the function. The classes vector, array, list, deque, set, and map are containers that are part of the Standard Template Library (STL). The components of the STL that work like pointers do on built-in arrays are called iterators. A class that has a pure virtual function is called an abstract class, while a class with no pure virtual functions is called a concrete class.

II. True or false

10 points

- T ☒ F You do not need to worry about numerical errors as long as you use double-precision floating point variables for real numbers because they have plenty of precision.
- ☒ T F Overflow errors are mainly a concern for integer variables.
- ☒ T F Class composition represents a "has-a" relationship while inheritance represents an "is-a" relationship.
- T ☒ F Designating a function "in-line" guarantees that the compiler will generate a copy of the function's code in every place where the function is called to avoid overhead of a function call.
- ☒ T F A copy constructor is called when an object is instantiated with another object of the same class as the initializer and when an object is passed by value to a function.
- ☒ T F Pure virtual functions allow different types of objects derived from the same base class to exhibit polymorphic behavior by defining the function in their own distinct ways.
- T ☒ F Arithmetic, such as addition and subtraction of iterators, can be done on iterators for any sequence container.
- ☒ T F In a map container, values are stored in sorted order based on the keys associated with the values stored in the map.
- T ☒ F In a programmer-defined class, it is necessary to overload the "=" operator to support assignment of one object of the class to another object of the class.
- ☒ T F I learned a lot this semester in ECE 2036 (Hint: The correct answer is true!)

III. Basic class creation

15 points

Write code to define a class named Complex to store a complex number with double values for its real and imaginary parts. The class should have a constructor with default values of 0.0 for both real and imaginary parts, and a destructor. Include a static variable that will track the number of Complex objects existing in the system at any time, include code in the appropriate member functions to update the static variable correctly, and initialize the static variable in the proper way. In addition to constructor and destructor, include these member functions: getReal(), setReal(double), getImag(), setImag(double), outputAsString(), and add(Complex). outputAsString() should use cout to output the number in the form a + bi. add(Complex) should return a Complex object that is the sum of the object being operated on and the Complex parameter. Follow all good programming practices you have learned during the semester in the creation of your class. You do not need any main() program, only the class definition.

using namespace std;

class Complex {

public:

Complex(double = 0.0, double = 0.0);

double getReal() const;

void setReal(double);

double getImag() const;

void setImag(double);

void outputAsString() const;

Complex add(Complex &) const;

~Complex();

private:

static int count;

double real;

double imag;

};

Complex::~Complex() {
 count--;
}

int Complex::count = 0;

Complex::Complex(double r, double i): real{r}, imag{i} {count++;}

double Complex::getReal() const {return real;}

void Complex::setReal(double r) {real = r;}

double Complex::getImag() const {return imag;}

void Complex::setImag(double i) {imag = i;}

void Complex::outputAsString() const {cout << real << " + " <<

Complex Complex::add(Complex & rhs) const {
 imag << "i" << endl;

Complex c;

c.real = real + rhs.getReal();

c.imag = imag + rhs.getImag();

return c;

}

IV. Program tracing with inheritance

15 points

Show the output of the following program.

```
#include <iostream>
using namespace std;

int n = 5;
int q = 2;

int calc(int x) {
    return x*x;
}

class Base {
    friend int main();
public:
    int calc(int m) {
        this->m = m-3;
        return m;
        k++;
    }
protected:
    static int k;
    int m;
};

class DerivedAndBase : public Base {
public:
    int calc(int & n) {
        m = n * 2;
        k++;
        return m;
    }
protected:
    int n;
};

class Derived : public DerivedAndBase {
};

int Base::k = 0;

int main() {
    int n = -3;
    int m = calc(n);
    cout << n << " " << m << endl;
    Base b;
    DerivedAndBase db;
    Derived d;
    int p = b.calc(q);
    cout << p << " " << b.m << endl;
    int o = 7;
    q = d.calc(o);
    cout << o << " " << q << endl;
    cout << Base::k << endl;
}
```

-3 9
2 -1
7 14
1

V. Operator overloading

15 points

For the following custom Array class, overload the "=" and "==" operators. The member function prototypes for the operators are provided for you in the class header below. For "=", avoid self assignment.

```
#include <iostream>
using namespace std;
```

```
class Array {
public:
```

```
    Array(int size = 10) {
        this->size = size;
        p = new int[size];
    }
```

```
    int getSize() const {return size;}
```

```
    const Array & operator=(const Array & right);
```

```
    bool operator==(const Array & right) const;
```

```
    ~Array() {delete [] p;}
```

```
private:
```

```
    int size;
    int * p;
};
```

```
const Array & Array::operator=(const Array & right){
    if (&right != this){
        if (size != right.getSize()){
            delete [] p;
            size = right.getSize();
            p = new int[size];
        }
        for (int i=0; i<size; ++i)
            p[i] = right.p[i];
    }
    return *this;
}
```

```
bool Array::operator==(const Array & right) const {
    if (size != right.getSize()) return false;
    for (int i=0; i<size; ++i)
        if (p[i] != right.p[i]) return false;
    return true;
}
```

VI. Templated functions

15 points

Write a templated function that has a vector containing any type of value as its parameter and swaps the first half of the vector with the last half of the vector, e.g. the vector of int's [3 3 9 -4 -7 11 -1 19 28 2] should become [11 -1 19 28 2 3 3 9 -4 -7]. The function should pass its vector parameter by reference so the vector in the calling routine is actually swapped after the function is executed. You will get 12 points for a correct solution that handles vectors with an even number of elements. To get 15 points, your function should handle both even-sized and odd-sized vectors. For an odd-sized vector, the middle element should remain unchanged after the swap, e.g. the vector [3 3 9 -4 -7 11 -1 19 28 2 -14] should become [-1 19 28 2 -14 11 3 3 9 -4 -7]. You do not need any main() program, only the function definition.

```
template <typename T>
void swap (vector<T> & con) {
    vector<T> conTemp(con.size());
    int middle = con.size()/2;
    int secondHalf = con.size()/2.0 + 0.5;
    for (int i=0; i<middle; i++) {
        conTemp[i] = con[secondHalf+i];
        conTemp[secondHalf+i] = con[i];
    }
    if (con.size() % 2 == 1)
        conTemp[middle] = con[middle];
    con = conTemp;
}
```

VII. Standard Template Library (STL)

20 points

Write code to fill up a vector of long's with the first 40 Fibonacci numbers. Then, find and output the first non-zero Fibonacci number that is divisible by 9 and its index in the vector (output "No number divisible by 9" if none of the 40 numbers is divisible by 9). Finally, output the 40 Fibonacci numbers in the vector. You should use STL constructs for all these operations, **not** indexing. To be specific, use `generate_n()` with a generator function to generate the numbers, use `find_if()` with a lambda expression to search for a number divisible by 9, use iterator arithmetic to calculate the index of the number divisible by 9, and use iterators with dereferencing to output the numbers in the vector. The Fibonacci numbers are defined as follows. Fibonacci(1) = 0, Fibonacci(2) = 1, and for $n > 2$:

$$\text{Fibonacci}(n) = \text{Fibonacci}(n-1) + \text{Fibonacci}(n-2)$$

Hints: It might be easier (and it is allowed) to set the first two numbers in the vector explicitly to 0 and 1 and then use the generator function for the last 38 numbers. Also, declare a vector of 40 long's at the beginning of your code to hold the numbers and use that vector with `generate_n()`, i.e. **do not** declare a 0-length vector and try to add values to it with `push_back()`.

```
#include <iostream>
#include <vector>

using namespace std;

long Fib() {
    static int m=0;
    static int n=1;
    int fib = m+n;
    m = n;
    n = fib;
    return fib;
}

int main() {
    vector<long> fib(40);
    auto it1 = fib.begin();
    *it1++ = 0;
    *it1++ = 1;
    generate_n(it1, 38, Fib);
    auto it3 = find_if(fib.begin()+1, fib.end(), [] (auto x) {return (x%9 == 0);});
    if (it3 == fib.end())
        cout << "\nNo number divisible by 9" << endl;
    else
        cout << "\nFirst number divisible by 9 is " << *it3 << " stored at index " << it3-fib.begin() << endl;
    int i=0;
    for (auto it2=fib.begin(); it2 != fib.end(); it2++) {
        cout << *it2 << " ";
        if (++i % 5 == 0) cout << endl;
    }
}
```